SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0509**

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison.* (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

Experiments

Code of Conduct:

I **N.AKSHAYA(192425129)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: N.AKSHAYA

Q. No	Understanding of Concepts (25%)	Experimental Design (30%)	Use of Tools (20%)	Analysis of Results (15%)	Documentation (10%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)				Total Marks (Out of 50)		

Annexure A
Experiments

Instructions: Perform the experiment using the laboratory performance rubric
Understanding of Concepts, Experimental Design, Use of Tools, Analysis of Results, Documentation.

1. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Answer:

1. Aim

To compare **Sobel and Canny edge detection** techniques using the same input image and analyze their effectiveness.

2. Understanding of Concepts

Sobel Edge Detection

Sobel is a gradient-based method that detects intensity changes in horizontal and vertical directions using kernels.

- Simple and fast
- Produces thicker edges
- More sensitive to noise

Canny Edge Detection

Canny is a multi-stage edge detection method involving:

1. Gaussian smoothing
2. Gradient calculation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking by hysteresis

- Produces thin edges
- Reduces noise
- Provides better edge localization

3. Experimental Design

Input: Dog

Tools: Python, OpenCV, NumPy, Matplotlib

Procedure:

1. Read the input image.
2. Convert it to grayscale.
3. Apply Gaussian blur.
4. Apply Sobel edge detection.
5. Apply Canny edge detection.
6. Display and compare the results.

4. Python Program

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("Dog.png", 0)

if img is None:

    print("Image not found!")

    exit()

blur = cv2.GaussianBlur(img, (5, 5), 0)

sx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)

sy = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)

sobel = cv2.magnitude(sx, sy)

canny = cv2.Canny(blur, 100, 200)

plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)

plt.imshow(img, cmap="gray")

plt.title("Original")

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(sobel, cmap="gray")
```

```
plt.title("Sobel")

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(canny, cmap="gray")

plt.title("Canny")

plt.axis("off")

plt.tight_layout()

plt.show()

print("\nSUMMARY")

print("Sobel  : Fast, simple, thicker edges")

print("Canny  : Thin, accurate, noise resistant")

print("Best for precise boundaries: Canny")
```

5. Results

6. Analysis

Feature	Sobel	Canny
Noise	High	Low
Edge thickness	Thick	Thin
Accuracy	Moderate	High
Speed	Fast	Slightly slower
Best use	Simple detection	Precise boundaries

7. Conclusion

Sobel is **simple and fast**, whereas Canny provides **better noise suppression and accurate edge localization**. Therefore, Canny is generally better for precise object boundary detection.

2. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

Answer:

1. Aim

To detect important **corner points** in an image using the Harris Corner Detection technique and analyze the detected corners.

2. Understanding of Concepts

Harris Corner Detection identifies points where image intensity changes significantly in two directions.

- High → Corner
- Low → Flat/edge region
- Useful for feature extraction and image matching.

3. Experimental Design

Input: Tree

Tools: Python, OpenCV, NumPy, Matplotlib

Procedure:

1. Read the image.
2. Convert it to grayscale.
3. Apply Harris Corner Detection.
4. Threshold the corner response.
5. Mark detected corners in **RED**.
6. Display and analyze the result.

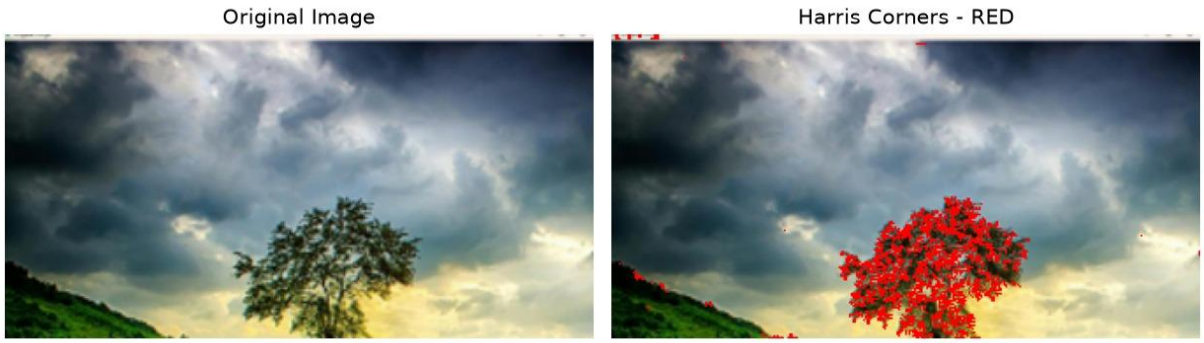
4. Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

path = r"C:\Users\aksha\Downloads\ITA0509\Experiments\Tree.png"
```

```
img = cv2.imread(path)

if img is None:

    print("Image not found!")

    exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)

corners = cv2.cornerHarris(gray, 2, 3, 0.04)

result = img.copy()

result[corners > 0.01 * corners.max()] = [0, 0, 255]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)

plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))

plt.title("Original Image")

plt.axis("off")

plt.subplot(1, 2, 2)

plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))

plt.title("Harris Corners - RED")

plt.axis("off")

plt.show()
```

```
print("\nSUMMARY")

print("Method: Harris Corner Detection")

print("Corner points are marked in RED")

print("Image size:", img.shape)
```

5. Results



6. Analysis

Feature	**Observation**
Flat region	Few corners
Textured region	More corners
Object corners	Clearly detected
Application	Feature extraction

7. Conclusion

Harris Corner Detection successfully identifies important corner points and is useful for **feature extraction, image matching, and object recognition**.

3. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Answer:

1. Aim

To detect **lines or circles** in an image using the **Hough Transform** and analyze the detected shapes.

2. Understanding of Concepts

The **Hough Transform** is a feature extraction technique used to detect geometric shapes such as **lines and circles**.

- Detects shapes even when parts are missing.
- Uses edge information to identify shapes.
- Useful in object detection and shape analysis.

3. Experimental Design

Input: cv
Tools: Python, OpenCV, NumPy, Matplotlib

Procedure:

1. Read the image.
2. Convert it to grayscale.

3. Apply Canny edge detection.
4. Apply Hough Transform.
5. Detect lines/circles.
6. Draw detected shapes in **RED**.
7. Display the result.

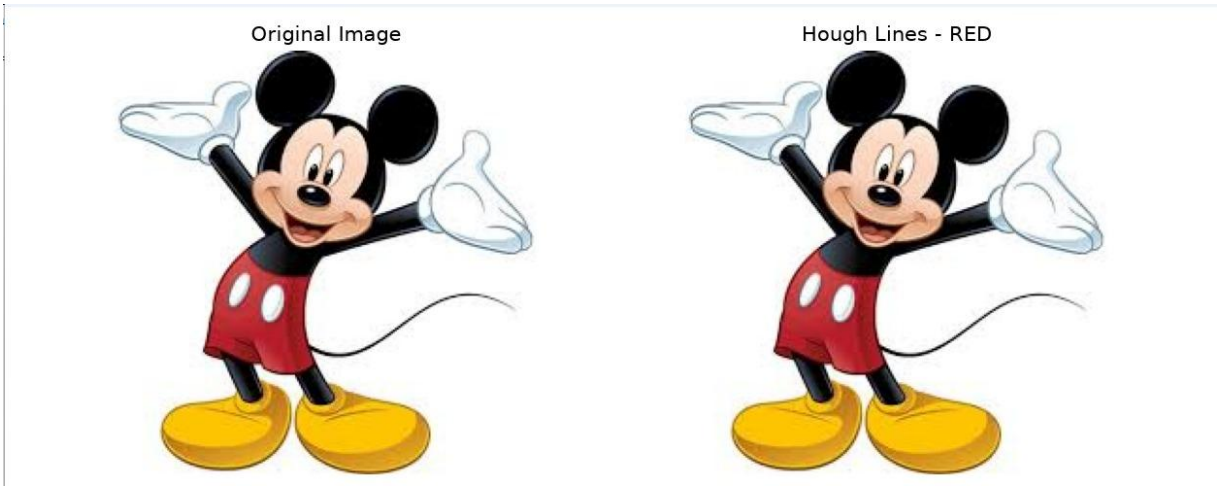
4. Python Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
path = r"C:\Users\aksha\Downloads\ITA0509\Experiments\cv.png"
img = cv2.imread(path)
if img is None:
    print("Image not found!")
    exit()
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLinesP(
    edges, 1, np.pi / 180,
    threshold=80,
```

```
    minLineLength=50,
    maxLineGap=10
)
result = img.copy()
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(result, (x1, y1), (x2, y2),
                 (0, 0, 255), 2)
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Hough Lines - RED")
plt.axis("off")
```

```
plt.tight_layout()

plt.show()

print("\nSUMMARY")

print("----------------------")

print("Method: Hough Transform")

print("Detected lines:", 0 if lines is None else len(lines))

print("Detected lines are shown in RED")
```

5. Results



6. Analysis

Parameter	**Observation**
Shape detected	Lines
Detection	Based on edges
Accuracy	Depends on threshold

Parameter	Observation
Visualization	RED lines
Application	Shape and object detection

7. Conclusion

The Hough Transform successfully detects prominent **lines** in the image and is useful for **shape detection, object recognition, and image analysis**.

4. Feature Matching using SIFT

Perform feature extraction and matching between two images using the SIFT algorithm. Explain the concept of feature matching and scale-invariant features. Use suitable software/tools to extract and match features, document the matching key points systematically, and analyze the robustness of SIFT under variations in scale, rotation, and illumination.

Answer:

1. Aim

To extract and match features between two images using the **SIFT (Scale-Invariant Feature Transform)** algorithm and analyze its performance under changes in scale, rotation, and illumination.

2. Understanding of Concepts

SIFT detects important keypoints and creates descriptors that are relatively invariant to:

- Scale
- Rotation
- Illumination changes

It is useful for **image matching, object recognition, and feature extraction**.

3. Experimental Design

Input: Tree cv

Tools: Python, OpenCV, NumPy, Matplotlib

Procedure:

1. Read two images.
2. Create a SIFT detector.
3. Detect keypoints and descriptors.
4. Match the descriptors using a BF Matcher.
5. Display the matched keypoints.
6. Analyze the matching results.

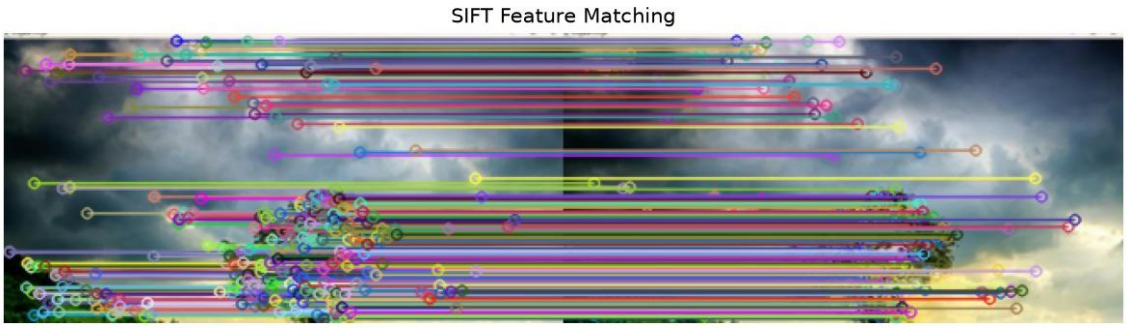
4. Python Program

```
import cv2

import matplotlib.pyplot as plt

path1 = r"C:\Users\aksha\Downloads\ITA0509\Experiments\tree.png"

path2 = r"C:\Users\aksha\Downloads\ITA0509\Experiments\tree.png"

img1 = cv2.imread(path1)

img2 = cv2.imread(path2)

if img1 is None or img2 is None:

    print("Image not found!")
```

```
    exit()

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray1, None)

kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher()

matches = bf.knnMatch(des1, des2, k=2)

good = []

for m, n in matches:

    if m.distance < 0.75 * n.distance:

        good.append(m)

output = cv2.drawMatches(

    img1, kp1, img2, kp2, good, None,

    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS

)

plt.figure(figsize=(12, 6))

plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))

plt.title("SIFT Feature Matching")
```

```
plt.axis("off")

plt.show()

print("\nSUMMARY")

print("----------------------")

print("Image 1 Keypoints:", len(kp1))

print("Image 2 Keypoints:", len(kp2))

print("Good Matches:", len(good))
```

5. Results



6. Analysis

Feature	**SIFT**
Scale changes	Robust
Rotation changes	Robust
Illumination changes	Relatively robust

Feature	SIFT
Feature matching	Accurate
Application	Object/image matching

7. Conclusion

SIFT successfully extracts and matches important features between two images. It is useful for **image matching and object recognition**, particularly when images have changes in scale or rotation.

5. PCA for Feature Reduction

Design and implement an experiment using Principal Component Analysis (PCA) for dimensionality reduction of extracted image features. Explain the importance of feature reduction in machine learning and computer vision applications. Apply PCA using appropriate tools, document the feature dimensions before and after reduction, and analyze its impact on computational efficiency and feature representation quality.

Answer:

1. Aim

To implement **Principal Component Analysis (PCA)** for reducing the dimensionality of image features and analyze its effect on feature representation.

2. Understanding of Concepts

PCA is a dimensionality reduction technique that transforms high-dimensional features into a smaller number of important components.

- Reduces feature dimensions
- Removes redundant information
- Improves computational efficiency
- Preserves important information

3. Experimental Design

Input: Tree

Tools: Python, OpenCV, NumPy, Scikit-learn, Matplotlib

Procedure:

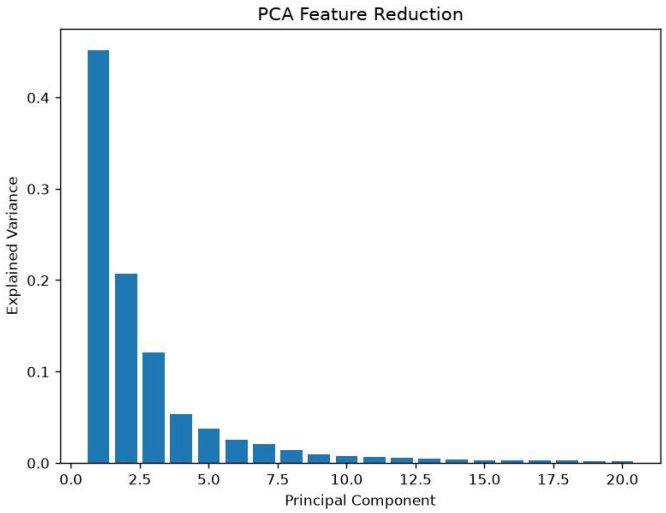
1. Read the image.
2. Convert it to grayscale.
3. Resize the image.
4. Flatten the image into features.
5. Apply PCA.
6. Reduce the number of features.
7. Compare dimensions before and after reduction.
8. Visualize the result.

4. Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

path = r"C:\Users\aksha\Downloads\ITA0509\Experiments\Flower.png"

img = cv2.imread(path, 0)

if img is None:

    print("Image not found!")

    exit()
```

```
img = cv2.resize(img, (100, 100))

features = img.reshape(1, -1)

pca = PCA(n_components=20)

reduced = pca.fit_transform(features)

print("\nPCA SUMMARY")

print("----------------------")

print("Original Dimensions :", features.shape)

print("Reduced Dimensions  :", reduced.shape)

plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)

plt.imshow(img, cmap="gray")

plt.title("Original Image")

plt.axis("off")

plt.subplot(1, 2, 2)

plt.bar(range(1, 21), abs(reduced[0]))

plt.title("PCA Features")

plt.xlabel("Principal Components")

plt.ylabel("Value")

plt.tight_layout()

plt.show()
```

5. Results



6. Analysis

Parameter	**Before PCA**	**After PCA**
Feature dimension	High	20
Redundant information	Higher	Reduced
Computation	Higher	Lower
Representation	Original	Compact

7. Conclusion

PCA successfully reduces the dimensionality of image features while retaining important information. It improves **computational efficiency** and provides a compact feature representation, making it useful in computer vision and machine learning applications.

RUBRICS FOR CALCULATION:

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**

Understanding of Concepts	**25**	**Demonstrates thorough understanding and effectively integrates multiple concepts/technologies**	**Good understanding with proper integration of most concepts**	**Basic understanding; limited or partial integration**	**Poor understanding; unable to integrate concepts**
Experimental Design	**30**	**Well-planned, systematic implementation with optimal solution**	**Correct implementation with minor issues**	**Basic implementation with errors or inefficiencies**	**Incorrect or incomplete implementation**
Use of Tools	**20**	**Effective and advanced use of tools/technologies with proper configuration**	**Appropriate use of tools with correct execution**	**Limited or inconsistent use of tools**	**Incorrect or minimal use of tools**
Analysis of Results	**15**	**Insightful analysis with accurate interpretation and validation of results**	**Good analysis with reasonable interpretation**	**Basic analysis with limited insights**	**Poor or incorrect analysis of results**
Documentation	**10**	**Clear, wellstructured documentation with diagrams, code, and results**	**Organized documentation with necessary details**	**Basic documentation with missing details**	**Poor or incomplete documentation**